

Path Planning Task Report



Name: Natalie Sameh Saad

Code: 2500617

Project: Path Planning task using Python.

The origin of the idea:

The core algorithm concept started with a simple sketch on paper. While drawing the track boundaries (blue cones on left, yellow cones on right), we realized that if we continuously connect each cone to the closest cone on the opposite side, it creates a “zig-zag” pattern across the track which is inspired by delaunay triangulation. By calculating the exact midpoint of each line of this zig-zag, we get a perfect set of anchor points in the center of the track. We knew these raw points will make a jagged path so we came up with the idea to apply a mathematical smoothing path at the end to round out the corners and make trajectory natural to drive the car.

Code Breakdown:

Here is how each part of the code works:

1. Settings & Basic Tools:

Settings: The code defines basic rules, like assuming the track is 2 meters wide (so half-width is 1 meter), generating exactly 8 meters of path, spacing points every 25 cm, and applying 15 smoothing passes.

Math Helpers: Simple functions to calculate the straight-line distance between two points using Pythagorean theorem, find the direction (using unit vector) and sort cones from closest to farthest.

2. Data Preparation:

First, the algorithm reads current heading and convert it into x,y direction vector using sine and cosine. Then, it divides the detected cones into two lists based on color: Blue for the left boundary and Yellow for the right boundary. Finally, sorting by distance so that the cones closest to the car are processed first.

3. Decision Making & Mathematical Logic:

Based on the cones we see, the code chooses one of three paths:

Case A: Both Sides are Visible (The Zig-Zag Method) as we see both blue and yellow cones, the code runs the `_both_sides` function.

First, it finds the closest blue-yellow pair. The line between them goes across the track. By mathematically rotating this line 90 degrees, it finds the track's forward direction. Then forming the Zig-Zag as it sorts all cones along this forward direction, it walks through the cones, always drawing an edge to the opposite cone that gives the shortest distance. Finally, it calculates the exact midpoint (x, y) of every edge. These midpoints become our central "anchors" and from these points the track starts to be found.

Case B: Only One Side is Visible (The Shift Method) as if the camera only sees one color (like blue cones on a sharp left turn), it runs the `_one_side` function.

The idea is to draw a line along the visible cones, then shift it sideways by half the track width (1 meter) to guess where the center is. To Calculate the Shift, for every cone, the code looks at the cone before it and the cone after it to find the direction of the curve. It rotates this direction 90 degrees and pushes the point 1 meter to the side.

The Single Cone Edge-Case: If the camera only sees exactly one cone, the code cannot look at neighboring cones to find the curve. Instead, it just takes the car's current heading, rotates it 90 degrees, and shifts the point 1 meter sideways.

Case C: No Cones are Visible (Go Straight). If the lists are completely empty, there are no anchors to calculate. The code simply relies on the car's current heading. It projects a line straight ahead based on the car's yaw angle, telling the car to keep driving in the same direction until new cones appear.

4. Path Building and Smoothing:

The final step transforms the raw anchor points into a clean, drivable path for the steering controller:

The Rough Line: The algorithm connects the car's position to the anchors, adding a final point straight ahead to guarantee a full 8-meter path (`PATH_LENGTH`).

Resampling: Steering systems need predictable data. The `_resample` function traces the rough line and places a new point exactly every 25 cm (`STEP = 0.25`), ensuring perfectly even spacing from start to finish.

Smoothing (Rounding Corners): The raw zig-zag anchors create sharp corners. To fix this, the code loops 15 times (`SMOOTHING_PASSES`), replacing each point with the average of its neighbors. This mathematical way gently pulls sharp edges inward, turning a jagged line into a perfectly smooth racing curve.

5. Testing & Custom Scenarios:

First, the algorithm was tested using 20 default scenarios that include different cone colors and numbers. To test more cases, 8 custom scenarios are tested and added them to the `scenario.py` file.